

Day 6: Pruning, distillation, and efficient design

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 5, 2026

- ▶ Quantization makes each value smaller. It does not change the number of values or the number of operations.
- ▶ Pruning removes weights. Distillation trains a smaller model. An efficient design starts with fewer operations.
- ▶ A method that looks good on paper can give no gain on your device. The gain depends on the hardware and on the runtime.
- ▶ In the lab you prune a CNN, distill a teacher into a student, quantize the best models, and select one model for each board of this course.

After this day you can:

- ▶ Apply unstructured and structured magnitude pruning.
- ▶ Explain why sparsity does not always reduce latency.
- ▶ Train a small student model with knowledge distillation.
- ▶ Select an optimization technique from a hardware constraint.

Part 1 Pruning

Exercise: predict the latency of a model with 80 percent sparsity on a CPU

Part 2 Knowledge distillation and low-rank methods

Exercise: calculate soft targets at two temperatures

Part 3 Efficient design and technique selection

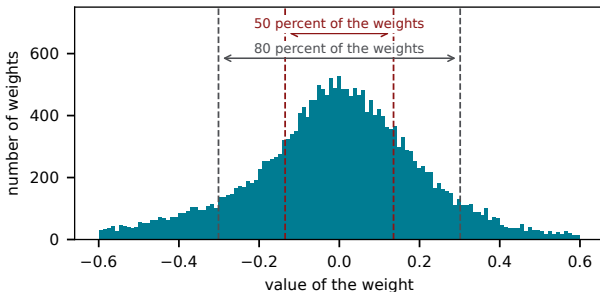
Exercise: select techniques for three cases

Three ways to a smaller model

Method	What it changes	Day
Quantization	The number of bits for each value	Day 4
Pruning	The number of values: it removes weights from a trained model	Part 1 of today
Distillation	The model: it trains a new, smaller model	Part 2 of today
Efficient design	The architecture: fewer operations from the start	Part 3 of today

- ▶ The four methods work together. A product often uses three of them on one model.
- ▶ Each method has a different effect on the three budgets: flash, RAM, and time.

Key point: Quantization changed how you store a value. Today you change how many values and how many operations the model has.



- ▶ A trained network has many weights near 0. Such a weight changes the output of its layer very little.
- ▶ Pruning sets these weights to exactly 0. The **sparsity** is the part of the weights that are 0.

Source: The weights of the example model of this part: a CNN for Fashion-MNIST with 23824 weights in its kernels.

The example model of this part

Layer	Output	Weights
Input: image 28×28 , 1 channel	$28 \times 28 \times 1$	none
Convolution 3×3 , 16 filters, then pooling	$14 \times 14 \times 16$	144
Convolution 3×3 , 32 filters, then pooling	$7 \times 7 \times 32$	4608
Convolution 3×3 , 64 filters, then global pooling	64	18 432
Fully connected, softmax	10	640

- ▶ 23 946 parameters with the biases. Accuracy on the test set: 89.1 percent.
- ▶ 1.92 million MACs. Peak of the activations: 15 680 values.
- ▶ LiteRT file in float32: 99 320 bytes. Latency on a laptop CPU with one thread: 66 microseconds.

Source: An experiment of this course with Fashion-MNIST (10 classes of clothes, images of 28×28). The lab of today uses the same model.

$$\hat{W} = M \odot W$$
$$M_{ij} = \begin{cases} 1 & \text{if } |W_{ij}| > \delta \\ 0 & \text{in all other cases} \end{cases}$$

- W The weights of a layer
- M The mask: 1 for a weight that stays, 0 for a weight that goes
- $\odot$ The product of each weight with its mask value
- δ The threshold

- ▶ The criterion is the magnitude: the absolute value of the weight.
- ▶ You select the sparsity, for example 80 percent. The threshold is then the magnitude that 80 percent of the weights are below.
- ▶ For the example model: $\delta = 0.135$ for 50 percent, and $\delta = 0.301$ for 80 percent.

Source: Formula: *Machine Learning Systems*, Volume I, chapter 10. Threshold values: the example model.

Criterion	The score of a weight or a filter	Cost
Magnitude	The absolute value, or the sum of the absolute values of a filter	None: the weights are enough
Gradient	The weight times its gradient: the change of the loss when the weight goes	One pass over some data
Activation	The mean output of a filter on real data: a filter that is almost always 0 goes	One pass over some data
Random	No score	None. It is the baseline for a comparison.

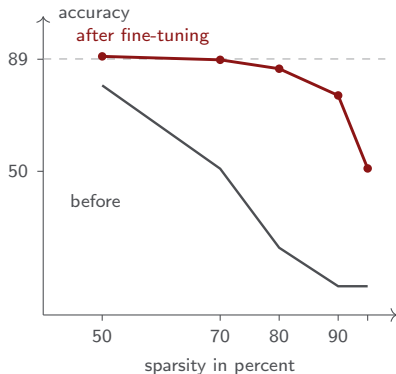
- ▶ The problem “find the best set of weights to remove” has no fast exact solution. All criteria are estimates.
- ▶ Magnitude is the standard: it is simple, and it works well with fine-tuning.

Layer	Weights	Local, 80 percent	Global, 80 percent
Convolution 1	144	80	51
Convolution 2	4608	80	71
Convolution 3	18 432	80	83
Fully connected	640	80	63

- ▶ **Local:** each layer has its own threshold. Each layer loses the same part of its weights.
- ▶ **Global:** one threshold for all layers. The table gives the sparsity in percent that each layer gets.
- ▶ With the global threshold, the first layer keeps half of its weights. It is small, and each of its weights is used at all positions of the image.
- ▶ The large layer loses most. This is the layer with the most redundancy.

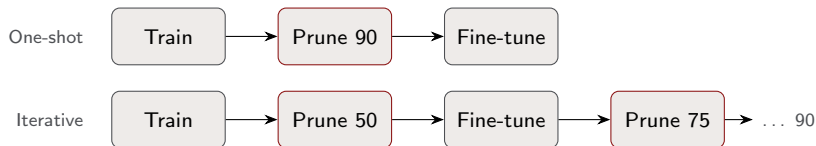
Source: Sparsity of each layer: the example model with a global sparsity of 80 percent.

Fine-tuning brings the accuracy back



- ▶ After the pruning, the accuracy falls: to 23.4 percent at a sparsity of 80 percent.
- ▶ **Fine-tuning:** train again for some epochs, with a small learning rate. The mask stays: a weight that is 0 stays 0.
- ▶ The other weights take the work of the removed weights.
- ▶ At 50 percent, the pruned model is better than the baseline: 90.0 against 89.1.

Source: The example model, global magnitude pruning, 2 epochs of fine-tuning. Accuracy in percent on the test set.



- ▶ **One-shot:** remove all weights in one step. It is fast. At a high sparsity, the model can lose too much to recover.
- ▶ **Iterative:** remove a part, fine-tune, and repeat. The model adapts in small steps. It costs one fine-tuning for each step.
- ▶ An example of the book with channel pruning of 27 percent: the iterative schedule loses 0.4 points, and the one-shot schedule loses 5 points.
- ▶ The example model of this part, to a sparsity of 90 percent with 3 epochs for each schedule: 78.4 percent (one-shot) and 77.3 percent (iterative). No gain: the result depends on the model.

Source: Example of the book: *Machine Learning Systems*, Volume I, chapter 10.

The hypothesis

A large network holds a small subnetwork that reaches almost the same accuracy when you train it alone, from its first random weights. This subnetwork is the “winning ticket”.

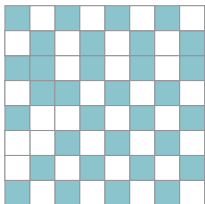
- ▶ The result explains why pruning works: most weights help the training, and the trained function needs only some of them.
- ▶ It is not a method to save training time: you find the ticket only after you trained the large network.
- ▶ For this course, the practical result is: start large, then prune. Do not start with a network that is too small.

How to find it

1. Train the network.
2. Prune the small weights.
3. Set the other weights back to their first values.
4. Repeat.

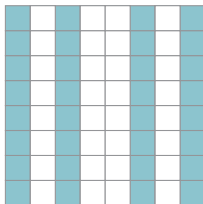
Source: Frankle and Carbin, “The Lottery Ticket Hypothesis”, 2019.

Granularity: what you remove



Unstructured: single weights

white = 0



Structured: complete filters

- ▶ **Unstructured:** each weight is a candidate. The tensor keeps its shape and gets zeros at irregular positions.
- ▶ **Structured:** a complete unit is a candidate: a neuron, a filter with its channel, or a layer. The tensor becomes smaller.
- ▶ The two figures have the same sparsity: 50 percent. The device sees two very different models.

Key point: Unstructured pruning has more freedom, so it keeps more accuracy. Structured pruning gives a model that normal hardware runs faster.

Sparsity	Weights not 0	LiteRT file	After gzip	Sparse format
None	23 824	99 320	90 877	not useful
50 percent	11 912	99 320	54 423	71 472
80 percent	4764	99 320	27 498	28 584
90 percent	2383	99 320	17 796	14 298

- ▶ All sizes are in bytes. The file does not change: a zero in `float32` needs 4 bytes, as each other value.
- ▶ A compression tool makes the file smaller. But the runtime of a microcontroller reads the model in place from the flash. It cannot read a compressed file.
- ▶ A sparse format stores each value with its position. The column gives 4 bytes for a value and 2 bytes for an index. The weights alone need 95 296 bytes in the dense format.

Source: File sizes: the example model. Sparse format: a calculation with 6 bytes for each weight that is not 0.

A dense kernel

- ▶ A loop over all weights, in order.
- ▶ The processor multiplies 4, 8, or 16 values in one instruction.
- ▶ A multiplication by 0 costs the same time as each other multiplication.

A sparse kernel

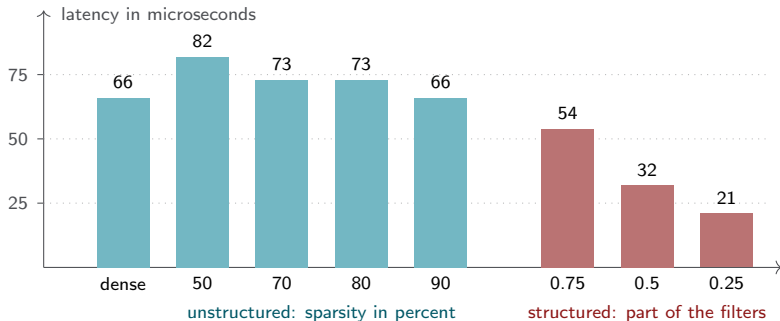
- ▶ It reads the position of each value first.
- ▶ It jumps in the memory. The cache and the wide instructions help less.
- ▶ It is faster only at a high sparsity, often above 80 to 90 percent.

- ▶ The kernels of LiteRT and of TensorFlow Lite Micro are dense kernels. Some runtimes have sparse kernels for some layer types.

Key point: Sparsity is a property of the model. Speed is a property of the model, the runtime, and the hardware together.

Source: Break-even sparsity: *Machine Learning Systems*, Volume I, chapter 10.

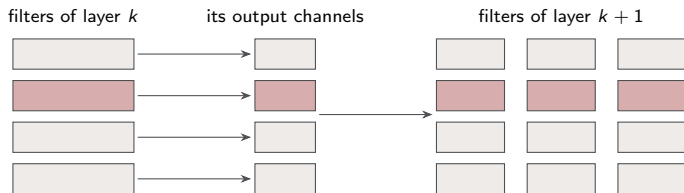
Measured: sparsity and latency



- ▶ Unstructured: the latency stays between 66 and 82 microseconds. The differences are noise of the measurement. The zeros give no gain.
- ▶ Structured: with half of the filters in each layer, the latency falls from 66 to 32 microseconds.

Source: The example model with LiteRT on a laptop CPU, one thread, median of 5 runs. The values say nothing about a board.

Structured pruning: remove a filter



- ▶ A filter of layer k makes one output channel. Without the filter, the channel does not exist.
- ▶ So each filter of layer $k + 1$ loses its weights for this channel.
- ▶ One removed filter makes two layers smaller, and it removes one feature map from the RAM.
- ▶ A usual criterion: the sum of the absolute values of the weights of the filter. Remove the filters with the smallest sum.

Key point: The result is a normal, smaller, dense model. Each runtime runs it with no change.

Structured pruning of the example model

Filters	Parameters	MACs	Accuracy	File	Latency	Peak
16, 32, 64	23 946	1 919 872	89.1	99 320	66	15 680
12, 24, 48	13 642	1 101 216	88.8	58 104	54	11 760
8, 16, 32	6218	508 352	85.6	28 408	32	7840
4, 8, 16	1674	141 280	76.2	10 232	21	3920

- ▶ Accuracy in percent after 2 epochs of fine-tuning. File in bytes, latency in microseconds, peak of the activations in values.
- ▶ With three quarters of the filters, the model loses 0.3 points and 41 percent of its file.
- ▶ With half of the filters, the MACs fall by a factor of 3.8. The latency falls by a factor of 2.1 only: the time has a part that does not depend on the MACs.
- ▶ The peak falls with the number of filters. Unstructured pruning does not change the peak.

Source: The example model with LiteRT on a laptop CPU, one thread.

The same accuracy, two different models

	Unstructured, 80 percent	Structured, half
Accuracy in percent	85.7	85.6
Weights not 0	4764	6152
File in bytes	99 320	28 408
Latency in microseconds	73	32
Peak of the activations	15 680	7840

- ▶ The unstructured model keeps fewer weights for the same accuracy. It has more freedom to select them.
- ▶ The structured model has half of the filters in each layer. It is better in each number that the device sees: flash, time, and RAM.

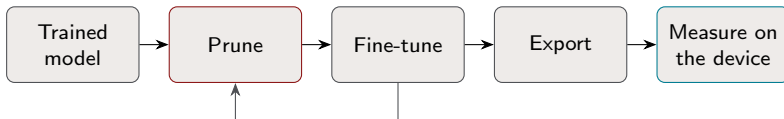
Key point: With dense kernels, only structured pruning pays. Unstructured pruning pays when the download of the file is the limit.

Source: The example model. Structured: 6152 weights and 66 biases.

Target	Unstructured	Structured
Microcontroller with TensorFlow Lite Micro	No gain	Flash, RAM, and time
Raspberry Pi with LiteRT	No gain in time	Flash, RAM, and time
A runtime with sparse kernels	Time, at a high sparsity	Time
The download of a model over a slow network	A smaller compressed file	A smaller file

- ▶ The table is the rule of this course for its two boards.
- ▶ Many papers report the sparsity or the number of parameters only. These numbers do not tell you the gain on your device.

Key point: Before you prune, ask: which kernels does my runtime have? Then measure the pruned model on the device.



- ▶ **Prune and fine-tune** a trained model in some steps. After each step, write the accuracy.
- ▶ **Export.** For structured pruning, build the smaller model and copy the weights that stay. A mask alone saves nothing: the tensor still has its full size.
- ▶ **Measure** the file, the latency, and the RAM of the exported model on the device.
- ▶ **Then quantize.** Pruning and quantization multiply: fewer values, and fewer bits for each value.

Key point: The pruned model of the training tool is not the model of the device. Only the exported file counts.

What pruning cannot do

- ▶ It keeps the architecture: the same layer types and the same connections.
- ▶ It cannot change the input size or the first feature map.
- ▶ A model that was small from the start has little to remove.
- ▶ When the target needs a different architecture, pruning is the wrong tool. Part 2 gives a method that trains a new, smaller model with the help of the large model.

The curve has a knee

- ▶ To a sparsity of 70 percent, the example model loses 0.3 points.
- ▶ From 80 to 90 percent, it loses 9 points.
- ▶ Stop before the knee.

Source: Accuracy values: the example model with unstructured pruning and 2 epochs of fine-tuning.

Exercise: predict the latency of a sparse model

A model runs with LiteRT on the processor of a Raspberry Pi. It uses the normal, dense kernels.

Parameters	2.0 million
MACs for one inference	300 million
File in float32	7.6 MB
Latency	60 ms

A colleague prunes the model with unstructured magnitude pruning to a sparsity of 80 percent. The accuracy stays almost equal. The colleague says: “The model is now 5 times as fast and 5 times as small.”

1. Predict the latency of the pruned model. Give the reason.
2. Predict the size of the model file. How can the file become smaller?
3. The target is 30 ms. Propose a pruning method that can reach it. Which part of the channels do you remove, if the MACs of a convolution grow with the square of the number of channels?
4. Which of the two methods makes the peak RAM smaller?

Answer: predict the latency of a sparse model

1. About 60 ms: no change. The dense kernels multiply each weight, also a weight that is 0. The model still does 300 million MACs. A sparse kernel gives a gain only at a high sparsity, and only if the runtime has one for the layers of the model.
2. 7.6 MB: no change. Each zero needs 4 bytes. A sparse format with 4 bytes for a value and 2 bytes for an index needs $400\,000 \times 6$ bytes: 2.3 MB. A compression tool also helps for the download, but not for the memory at run time.
3. Structured pruning: remove complete filters. For half of the time, you need about half of the MACs. With $c^2 = 0.5$, the part of the channels that stays is $c = 0.71$. Remove about 30 percent of the channels of each layer. The example of this part shows that the time falls less than the MACs, so plan for 35 to 40 percent, and measure.
4. Structured pruning. Fewer channels give smaller feature maps. Unstructured pruning keeps the shape of each tensor, so the peak RAM does not change.

After each of the two methods, measure the accuracy again.

Part 1 Pruning

Exercise: predict the latency of a model with 80 percent sparsity on a CPU

Part 2 Knowledge distillation and low-rank methods

Exercise: calculate soft targets at two temperatures

Part 3 Efficient design and technique selection

Exercise: select techniques for three cases

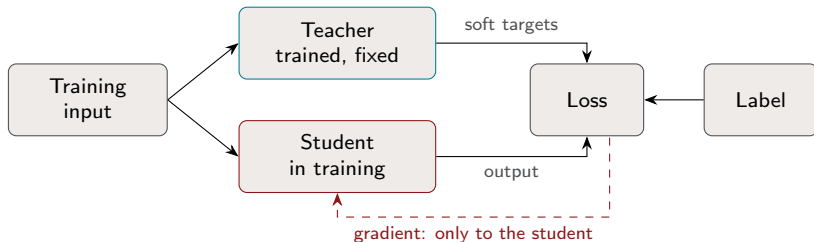
Pruning (Part 1)

- ▶ Starts from the trained model and removes parts.
- ▶ The result has the same layer types and the same connections.

Distillation (this part)

- ▶ Trains a new model that you design for the device.
- ▶ The large model helps in the training. It does not go to the device.
- ▶ The large model is the **teacher**. The small model is the **student**.
- ▶ The student can have a different depth, a different width, and different layer types.
- ▶ The result is a normal dense model. Each runtime runs it with no change.

Key point: Distillation moves knowledge from one model to a different model. It does not move weights.



- ▶ The teacher and the student get the same input.
- ▶ The student learns from two sources: the label of the dataset, and the output of the teacher.
- ▶ The training changes only the student.
- ▶ After the training, the device runs the student alone.

Class	Label	Teacher
Shirt	1	0.909
Pullover	0	0.070
T-shirt	0	0.011
Coat	0	0.010
Trouser	0	0.000
5 other classes	0	0.000

- ▶ The label says one thing: “this is a shirt”.
- ▶ The teacher says more: “a shirt, a little like a pullover, and not like a trouser”.
- ▶ The small values tell which classes look similar. The label cannot tell this.

Key point: The output of the teacher for one image holds more information than its label. The student gets this information for each training image.

Source: The output of the example model of Part 1 for one test image of Fashion-MNIST.

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

	Shirt	Pullover	T-shirt	Coat	Trouser	Others
$T = 1$	0.909	0.070	0.011	0.010	0.000	0.000
$T = 2$	0.658	0.183	0.072	0.069	0.009	0.008
$T = 4$	0.403	0.212	0.134	0.131	0.048	0.072
$T = 8$	0.250	0.181	0.144	0.142	0.086	0.196

- ▶ z_i is the logit of class i : the output before the softmax. $T = 1$ is the normal softmax.
- ▶ A higher T makes the distribution flatter. The order of the classes stays, and the small values become visible.

Source: The same test image. The column "Others" is the sum of the 5 other classes.

$$L = \alpha \cdot L_{\text{hard}} + (1 - \alpha) \cdot T^2 \cdot L_{\text{soft}}$$

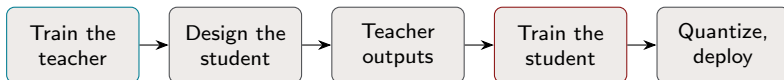
L_{hard}	The cross-entropy between the label and the output of the student at $T = 1$. It is the normal loss.
L_{soft}	The difference between the output of the teacher and the output of the student, the two at the same temperature T . The measure is the Kullback-Leibler divergence.
α	The weight of the label, from 0 to 1. A typical value is 0.1: the teacher counts most.
T^2	A correction. The gradient of the soft part becomes smaller by the factor $1/T^2$. The factor T^2 keeps the two parts in balance.

► The temperature is a tool of the training. On the device, the student uses the normal softmax, or no softmax.

Source: Hinton et al., “Distilling the Knowledge in a Neural Network”, 2015; *Machine Learning Systems*, Volume I, chapter 10.

```
def distillation_loss(T, alpha):  
    def loss(y_true, z_student):  
        labels = ops.cast(y_true[:, 0], "int32")    # the label  
        z_teacher = y_true[:, 1:]                  # the teacher logits  
        hard = keras.losses.sparse_categorical_crossentropy(  
            labels, z_student, from_logits=True)  
        p_teacher = ops.softmax(z_teacher / T)  
        log_p_student = ops.log_softmax(z_student / T)  
        soft = ops.sum(p_teacher * (ops.log(p_teacher + 1e-9)  
                        - log_p_student), axis=-1)  
        return alpha * hard + (1 - alpha) * T * T * soft  
    return loss
```

- ▶ The teacher runs one time over the training set. Its logits go into the target array, next to the label.
- ▶ The student has no softmax layer in the training: it gives logits.
- ▶ The training loop is the normal loop: `student.fit(x, y)`.



- ▶ **The teacher** is the best model that you can train. Its size is not important: it never goes to the device.
- ▶ **The student** comes from the budgets of the device: flash, RAM, and time.
- ▶ **The teacher outputs** need no labels. You can use more data than the labelled training set.
- ▶ **Always train the same student with the labels only.** This is the baseline. The distillation must be better.

Key point: Distillation costs training time: one run of the teacher over the data, and one training of the student. It costs nothing on the device.

Model	Parameters	Accuracy
Teacher: 32, 64, and 128 filters	93 962	90.3
Student: 8, 16, and 32 filters, labels only	6218	86.9
Student, distillation with $T = 1$	6218	87.1
Student, distillation with $T = 2$	6218	86.6
Student, distillation with $T = 4$	6218	85.8

- ▶ The student has 15 times fewer parameters than the teacher. It trains with all 60 000 labelled images.
- ▶ The gain of the distillation is 0.2 points at $T = 1$. The three runs of one setting differ by 0.4 points: this is no clear gain.

Key point: With many labels, a very small student is limited by its own size. The teacher cannot help it much.

Source: An experiment of this course: $\alpha = 0.5$, mean of three runs, accuracy in percent.

Training data of the student	Accuracy
1000 labelled images, labels only	77.4
The same 1000 images, distillation with $T = 2$	78.9
60 000 images with no label, outputs of the teacher only	85.3
For comparison: 60 000 labelled images, labels only	86.9

- ▶ With 1000 labelled images, the soft targets give 1.5 points: each image carries more information.
- ▶ The large gain comes from the third row. The teacher gives a target for each image that has no label: 7.9 points more.
- ▶ A label costs the time of a person. Raw sensor data costs almost nothing.

Key point: For an edge product, the strongest use of a teacher is this: it turns raw data from the field into training data for the small model.

Source: The same experiment. The teacher learned from all labelled images. Mean of three runs.

	All labels	1000 labels
Labels only	86.9	77.4
$T = 1$	87.1	78.4
$T = 2$	86.6	78.9
$T = 4$	85.8	77.5

- ▶ For this teacher, the best temperature is 1 or 2.
 - ▶ The mean of its largest probability is 0.921. Its outputs are soft at $T = 1$ already.
 - ▶ A teacher that gives 0.999 for almost each image needs a higher temperature.
-
- ▶ Many texts name a temperature of 4 or 5 as the typical value. The table shows that such a value is not a rule.
 - ▶ Find T and α with a small search: three temperatures, two values of α , and the same student each time.

Source: The same experiment: $\alpha = 0.5$, mean of three runs, accuracy in percent.

The student

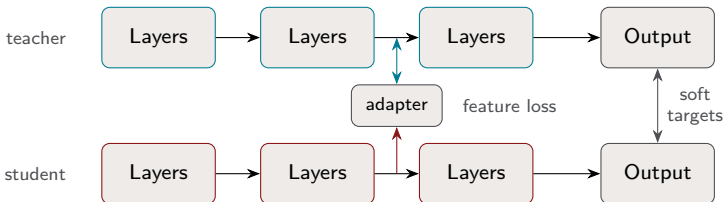
- ▶ A student that is too small cannot hold the function of the teacher. The gain becomes small.
- ▶ A very large gap between teacher and student can make the result worse.
- ▶ The data of the distillation must be like the data of the device. The student learns the teacher only where it sees examples.
- ▶ The values of T and α come from experiments. No rule gives them.

Key point: Distillation is not a compression with no loss. The student is limited by its own size, by the teacher, and by the data.

The teacher

- ▶ The student copies the errors of the teacher also.
- ▶ A teacher that is too sure gives soft targets near the labels. They carry little.

Source: *Machine Learning Systems*, Volume I, chapter 10.



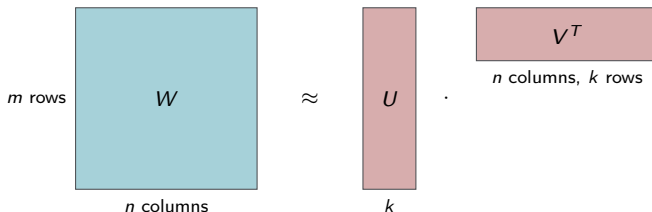
- ▶ The student also learns to copy a feature map inside the teacher, not only its output.
- ▶ The two feature maps have different numbers of channels. A small adapter layer, for example a convolution of 1×1 , makes the shapes equal. The adapter is not in the deployed model.
- ▶ The loss is the mean squared difference between the two feature maps.
- ▶ It helps most for deep students and for tasks with a small output, for example two classes: the output alone says little.

	Distillation	Pruning
Architecture	You design it	The architecture of the trained model
Training cost	One run of the teacher, and a complete training of the student	Some epochs of fine-tuning
Result	A dense model	Dense (structured), or with zeros (unstructured)
Works on normal kernels	Yes	Only the structured result
Typical use	A change to a different model family	A model that is some percent too large

- ▶ A known result: DistilBERT has 40 percent fewer parameters than BERT, is 60 percent faster, and keeps 97 percent of its language performance.

Source: Comparison and DistilBERT numbers: *Machine Learning Systems*, Volume I, chapter 10.

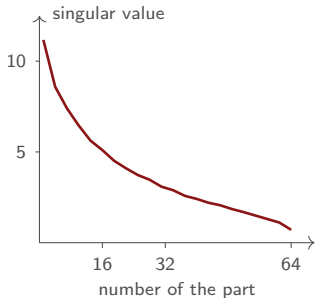
Low-rank factorization: the idea



$$W \approx U V^T \quad m \cdot n \text{ values become } k \cdot (m + n) \text{ values}$$

- ▶ A weight matrix with m rows and n columns becomes the product of two thin matrices. k is the rank.
- ▶ The layer $y = Wx$ becomes two layers: $y = U(V^T x)$. The device never calculates W .
- ▶ The method saves values only when $k < \frac{m \cdot n}{m + n}$.

Source: *Machine Learning Systems*, Volume I, chapter 10.



- ▶ The singular value decomposition (SVD) writes W as a sum of parts. Each part has a singular value: its weight in the sum.
 - ▶ Keep the k parts with the largest singular values. This is the best approximation of W with the rank k .
 - ▶ If the singular values fall fast, a small k is enough.
- ▶ The figure shows the third convolution of the example model as a matrix of 288×64 . Its singular values fall slowly: from 11.2 to 0.7.

Source: Singular values: the example model of Part 1. Method: the Eckart-Young theorem.

Rank k	Values of the factors	Against 18 432	Accuracy
64 (full)	22 528	122 percent	89.1
48	16 896	92 percent	89.0
32	11 264	61 percent	85.5
16	5632	31 percent	78.7
8	2816	15 percent	51.7

- ▶ Only the third convolution changes. It is the matrix of 288×64 with 18 432 weights.
- ▶ The method saves values only below a rank of 52:
 $18\,432 / (288 + 64) = 52.4$.
- ▶ A small layer of a small model is not low rank: each of its filters does a different job.
- ▶ Low-rank methods pay for large layers with redundancy, for example the layers of a language model.

Source: An experiment of this course with the example model of Part 1. Accuracy in percent, with no fine-tuning.

Matrix	Values
4096×4096	
Dense	16 777 216
Rank 128	1 048 576
Dense, float32	64 MB
Rank 128, float32	4 MB

- ▶ The factors need 16 times fewer values and 16 times fewer MACs.
- ▶ This is true only if the matrix has this redundancy. You must measure the accuracy.

- ▶ One layer becomes two layers. The device needs one more activation tensor with k values and one more kernel call.
- ▶ A convolution of 3×3 becomes a convolution of 3×3 with k filters and a convolution of 1×1 .
- ▶ After the factorization, fine-tune the model, as after pruning.

Source: Example of the matrix: *Machine Learning Systems*, Volume I, chapter 10.

	Structured pruning	Distillation	Low rank
Start	A trained model	A trained teacher	A trained model
The result	The same model with fewer filters	A new model that you design	The same model with two thin layers for one layer
Training	Fine-tuning	A complete training	Fine-tuning
Best for	A model that is somewhat too large	A large step in size	Large layers with redundancy

- The three methods give a dense model. After each of them, the quantization of Day 4 gives a factor of 4 more in the flash.

Key point: Compare the methods on one plot: accuracy against size and against latency. The lab does this.

Exercise: soft targets at two temperatures

A teacher gives these logits for one image:

Class	Shirt	Pullover	Coat	Sandal
Logit z	4.0	2.0	0.0	-2.0

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Values: $e^4 = 54.60$, $e^2 = 7.39$, $e^1 = 2.72$, $e^{0.5} = 1.65$, $e^0 = 1$, $e^{-0.5} = 0.61$, $e^{-2} = 0.14$.

1. Calculate the four probabilities for $T = 1$.
2. Calculate the four probabilities for $T = 4$.
3. Compare the classes “Coat” and “Sandal” in the two results. What does the student learn at $T = 4$ that it cannot learn at $T = 1$?
4. What is the result for a very large T ? Is this useful?

Answer: soft targets at two temperatures

1. $T = 1$: the sum is $54.60 + 7.39 + 1.00 + 0.14 = 63.13$.
2. $T = 4$: the logits become 1.0, 0.5, 0.0, and -0.5 . The sum is $2.72 + 1.65 + 1.00 + 0.61 = 5.98$.

	Shirt	Pullover	Coat	Sandal
$T = 1$	0.865	0.117	0.016	0.002
$T = 4$	0.455	0.276	0.167	0.102

3. At $T = 1$, the two classes have 0.016 and 0.002: almost no signal in the loss. At $T = 4$, they have 0.167 and 0.102. The student now learns that a shirt is more like a coat than like a sandal. The order of the classes is the same in the two rows.
4. Each class gets 0.25. All information is lost: the target is the same for each image. A very high temperature is not useful.

Part 1 Pruning

Exercise: predict the latency of a model with 80 percent sparsity on a CPU

Part 2 Knowledge distillation and low-rank methods

Exercise: calculate soft targets at two temperatures

Part 3 Efficient design and technique selection

Exercise: select techniques for three cases

The measured problem	The answer in the architecture	Example
The complete model is too large	Scale the depth, the width, and the image size together	EfficientNet
Some layers do too many operations	Replace them with cheaper layers: depthwise and pointwise	MobileNet
The activations do not fit the RAM	Smaller feature maps in the first layers	MCUNet
Good in theory, slow on the device	Put the measured latency of the device into the design	MobileNetV3

- ▶ Pruning, distillation, and quantization repair a model that exists. An efficient design does not make the problem.

Key point: If the architecture ignores the device, compression treats only the symptoms.

Source: Table: *Machine Learning Systems*, Volume I, chapter 10.

$$\text{MACs of one convolution} = H \cdot W \cdot k^2 \cdot C_{in} \cdot C_{out}$$

Control	Changes	Effect on the MACs
Image size	H and W of each feature map	With the square of the side
Width	C_{in} and C_{out} of each layer	With the square of the width
Depth	The number of layers	With the depth
Layer type	The factor $k^2 \cdot C_{in} \cdot C_{out}$	Depthwise and pointwise: about 8 times fewer for $k = 3$

- H and W are the size of the output map, k is the kernel size, and C is a number of channels.

Key point: A model has four controls for its cost. An efficient design sets all four for the device.

Model	Parameters	MACs	Accuracy	File	Latency	Peak
Baseline	23 946	1 919 872	89.1	99 320	71	15 680
Pruned	6218	508 352	85.6	28 408	26	7840
Separable	3946	356 576	86.3	20 872	27	15 680

- ▶ Baseline: 16, 32, and 64 filters. Pruned: 8, 16, and 32 filters. Separable: the baseline with a depthwise layer and a pointwise layer in place of the second and the third convolution, trained from the start.
- ▶ It has 6 times fewer parameters than the baseline and loses 2.8 points. It is smaller and more accurate than the pruned model.
- ▶ Its peak does not change: the feature maps have the same size. The pruned model has half of the peak.

Source: An experiment of this course. Accuracy in percent, file in bytes, latency in microseconds on a laptop CPU, peak in values. The latency values of Part 1 differ by the noise of a second run.

Model	Year	The new idea
MobileNetV1	2017	Depthwise separable convolutions in each layer. Two controls: the width multiplier and the resolution.
MobileNetV2	2018	The inverted residual block: narrow, wide, narrow, with a skip connection between the narrow tensors (Day 5).
MobileNetV3	2019	The blocks and the widths come from a search with the measured latency of a phone.

- The family is the standard start for vision on a small device. You select two numbers: the width multiplier and the image size.

Key point: Take a known efficient family, and set its controls for your budgets. This is faster than a new design, and it is usually better.

Source: Howard et al., 2017; Sandler et al., 2018; Howard et al., 2019.

$$d = \alpha^\phi \quad w = \beta^\phi \quad r = \gamma^\phi$$

d Factor for the depth
 w Factor for the width
 r Factor for the image size
 ϕ The size step that you select

- ▶ The MACs grow with $d \cdot w^2 \cdot r^2$.
- ▶ A larger image needs more layers and more channels to use its detail. So the three dimensions must grow together.
- ▶ The paper uses $\alpha = 1.2$, $\beta = 1.1$, and $\gamma = 1.15$. Each step of ϕ gives about two times the MACs.

Model	Parameters	Operations	ImageNet accuracy
ResNet-50	26 million	4.1 billion	76.0 percent
EfficientNet-B0	5.3 million	0.39 billion	77.1 percent

Source: Tan and Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks", 2019.

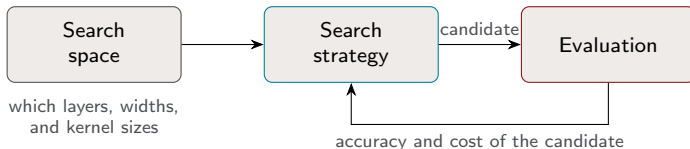
Fewer operations are not always less time

Against the baseline	MACs	Latency
Pruned to 8, 16, 32 filters	3.8 times fewer	2.7 times shorter
Separable layers	5.4 times fewer	2.6 times shorter
Baseline in <code>int8</code>	the same	1.6 times longer

- ▶ The time has parts that the MACs do not count: the memory access, the start of each kernel, and the copy of the input.
- ▶ On this laptop processor, the `int8` kernels are slower than the float kernels. On a microcontroller with no float unit, the result is the opposite.
- ▶ The book reports cases where the operation count and the latency differ by a factor of 3 to 5.

Key point: The operation count is an estimate for the design. The latency on the device is the result.

Source: The example model with LiteRT on a laptop CPU, one thread. Factor of 3 to 5: *Machine Learning Systems*, Volume I, chapter 10.



- ▶ A program designs the architecture. It tests many candidates and keeps the best.
- ▶ **Search space:** the building blocks and their options, for example a block with a kernel of 3, 5, or 7 and an expansion of 3 or 6.
- ▶ **Search strategy:** how the program selects the next candidate.
- ▶ **Evaluation:** the accuracy of the candidate, and its cost.

Key point: The search needs a trained model for each candidate. The evaluation is the expensive step.

Source: *Machine Learning Systems*, Volume I, chapter 10.

Strategy	How it works	Reported cost
Reinforcement learning	A controller proposes a model and gets its accuracy as the reward	22 400 GPU-days for 12 800 models
Evolution	A population of models. Change the best models and select again.	3150 GPU-days
Gradient (DARTS)	All options are in one large network with weights for each option	1 to 4 GPU-days

- The first searches trained each candidate from the start. Weight sharing trains one large network one time, and each candidate uses a part of it. The cost falls by a factor of about 1000.

Key point: A search is for a product with millions of devices. All others use the models that the searches found.

Source: *Machine Learning Systems*, Volume I, chapter 10.

$$\text{score} = \text{accuracy} \times \left(\frac{\text{latency}}{\text{target latency}} \right)^{\beta} \quad \beta < 0$$

- ▶ The score has two parts: the accuracy and the latency on the target device.
- ▶ A candidate that is slower than the target loses score. A candidate that is faster wins a little.
- ▶ The latency is a measurement on the device, or a table with the measured time of each block. The search then prefers layers that are fast on this device.

A reported result

A search with the latency of a phone in the score found models that are 1.8 times as fast as MobileNetV2, with a higher accuracy.

Source: Score and result of MnasNet (Tan et al., 2019), from *Machine Learning Systems*, Volume I, chapter 10.

The model: TinyNAS

- ▶ First it fits the search space to the device: it selects the image size and the width for the RAM and the flash.
- ▶ Then it searches a model in this space.

The runtime: TinyEngine

- ▶ It generates code for the layers of this one model only.
- ▶ It plans the memory for the complete model, not for each layer alone.

- ▶ The limit of the search is the peak activation memory of Day 5, not only the size of the weights.
- ▶ The model and the runtime are designed together. A better memory plan of the runtime permits a larger model.
- ▶ The result: more than 70 percent ImageNet accuracy on a commercial microcontroller.

Source: Lin et al., "MCUNet: Tiny Deep Learning on IoT Devices", 2020.

Each technique and the three budgets

Technique	Flash	RAM (peak)	Latency
Quantization to int8	About 4 times smaller	4 times smaller	Depends on the kernels
Unstructured pruning	No gain in place	No gain	No gain with dense kernels
Structured pruning	Smaller	Smaller	Shorter
Distillation	Smaller	Smaller, with smaller maps	Shorter
Separable layers	Smaller	No gain	Shorter
A smaller image	No gain	Much smaller	Shorter

Key point: Start from the budget that fails. Then select the technique with an entry in that column.

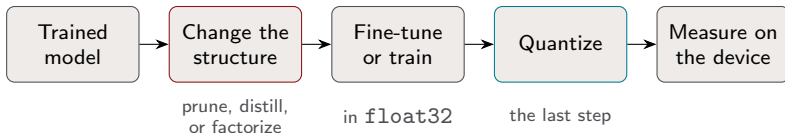
Source: The summary of Days 4, 5, and 6 for a device with dense kernels.

Step	Bytes	Factor	Accuracy
Baseline, float32	99 320	1.0	89.1
Pruned to 12, 24, 48 filters	58 104	1.7	88.8
Then int8	20 216	4.9	88.8
Pruned to 8, 16, 32 filters	28 408	3.5	85.6
Then int8	12 040	8.2	85.9

- ▶ Pruning removes values. Quantization makes each value smaller. The two factors multiply.
- ▶ The first chain gives a file that is 4.9 times smaller, with a loss of 0.3 points.
- ▶ The int8 file of a very small model is not 4 times smaller: the description of the graph does not shrink (Day 4).

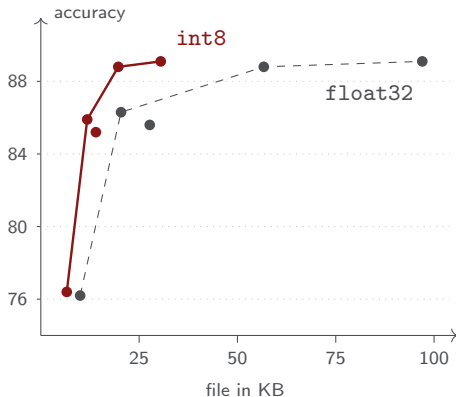
Source: The example model: structured pruning with 2 epochs of fine-tuning, full integer quantization with 200 calibration images.

The order: prune, distill, quantize



- ▶ The structural methods need training with gradients. Training works best in `float32`.
- ▶ Quantization needs the final weights and the final activations: the calibration measures their ranges. A change of the structure after it makes the ranges wrong.
- ▶ If the `int8` model loses too much, the last training step is quantization-aware training (Day 4).
- ▶ After each step, measure the accuracy. Then you know which step caused a loss.

Key point: First decide what the model calculates. Then decide how it stores the numbers.



- ▶ Each point is one model: the baseline, three pruned models, and the separable model.
- ▶ The line connects the models that no other model beats in size and in accuracy.
- ▶ In float32, the separable model is on the line. In int8, it is below the line: it loses 1.1 points in the quantization.

Key point: Draw the curve for the format of the device. Then take the point on the line that fits your budget.

Source: The example model: file size of the LiteRT file and accuracy in percent on the test set.

1. **Write the budgets** of the device: flash, RAM, latency, and the smallest accuracy that the product accepts.
2. **Measure the baseline** in `float32` on the device, or estimate it: file size, peak of the activations, MACs.
3. **Quantize first.** Post-training quantization needs seconds, and it helps two or three budgets.
4. **Find the budget that still fails.** Select the technique for this budget from the table.
5. **Apply the cheapest technique first:** a smaller image or a smaller width, then structured pruning, then distillation, then a new architecture.
6. **Measure all budgets again** after each step. A step can repair one budget and break a different one.

Key point: The selection is a loop of measurements, not one decision. Stop at the first model that meets all budgets.

Part	Time	Work
Quiz	20 min	The quiz of Week 1
A	45 min	Prune the example CNN at several sparsity levels, and fine-tune it. Compare unstructured and structured pruning.
B	45 min	Distill a teacher into a smaller student. Compare with the same student with labels only.
C	40 min	Quantize the best models with the method of Day 4. Plot the accuracy against the size and against the latency. Select one model for each board.

- ▶ The lab needs no board. It runs on the laptop.
- ▶ The deliverable is the plot with at least six models, and a Decision Log that names the constraint that decides.

Key point: The plot is the tool of the engineer. A single number for one model says nothing about the alternatives.

Exercise: select techniques for three cases

One vision model, three products. The model in float32:

Weights	1.2 million parameters: 4.6 MB
Peak of the activations	600 000 values: 2.3 MB
Latency on the target processor	400 ms
Image size	160×160

1. **Flash limit.** The device has 1.5 MB of flash for the model. The RAM and the latency are no problem.
2. **RAM limit.** The device has 256 KB of RAM for the activations. The flash is no problem.
3. **Latency limit.** The product needs 150 ms. The device has a float unit and no optimized int8 kernels.

For each case: name the first technique, calculate its effect, and say if a second technique is necessary. Use the factors of this day: 4 for int8, and the square of the side for the image size.

1. **Flash.** Quantize to int8: $4.6/4 = 1.15$ MB. This is below 1.5 MB. No second technique is necessary. Check the accuracy.
2. **RAM.** Quantize to int8: the peak becomes 600 000 bytes, or 586 KB. This is still above 256 KB. Then use a smaller image. With 96×96 , the factor is $(96/160)^2 = 0.36$: 211 KB. This fits. Structured pruning of the first layers is the alternative. Unstructured pruning and separable layers do not help.
3. **Latency.** The target is $150/400 = 0.375$ of the time. Quantization does not help on this device. Reduce the MACs: the image size 96×96 gives a factor of 0.36, or structured pruning to 60 percent of the channels gives $0.6^2 = 0.36$. The time falls less than the MACs, so plan a margin, and measure. If the accuracy is then too low, distill into the smaller model with the large model as the teacher.

One technique repairs cases 2 and 3 together: the smaller image. It is also the cheapest. Measure the accuracy first.

- ▶ Unstructured pruning makes the model file smaller. Only structured pruning makes a normal runtime faster.
- ▶ Distillation trains a small student with the soft outputs of a large teacher. The temperature controls how much the small probabilities count.
- ▶ An efficient design decreases the operations and the activations before the training starts.
- ▶ The constraint selects the technique: flash, RAM, or latency. The usual order is prune, distill, quantize.

Key point: Measure each technique on the target device. A smaller number of parameters is not a smaller latency, and it is not a smaller peak RAM.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 10 “Model Compression”, sections “Structural Optimization”, “Architectural Efficiency”, and “Technique Selection”.

Papers

- ▶ “Learning both Weights and Connections for Efficient Neural Networks”, Han et al., 2015.
- ▶ “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks”, Frankle and Carbin, 2019.
- ▶ “Distilling the Knowledge in a Neural Network”, Hinton et al., 2015.
- ▶ “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”, Tan and Le, 2019.
- ▶ “MCUNet: Tiny Deep Learning on IoT Devices”, Lin et al., 2020.

Course

- ▶ MIT 6.S940 “TinyML and Efficient Deep Learning Computing”, for more depth: <https://efficientml.ai>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.